Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Исследование зависимости точности вычислений от порядка сложения членов на примере ряда Тейлора»

Выполнил:
студент группы 3825Б1ПМии1
Первушин И. Д.

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение 3
Постановка задачи 4
Руководство пользователя 5
Описание программной реализации 6
Результаты экспериментов 8
Заключение 10
Литература 11
Приложение 12

Введение

Я создал программу, которая позволяет вычислять значения некоторых элементарных функций при помощи ряда Тейлора, складывая члены в различных направлениях и сравнивая получившееся значения с эталонным из математической библиотеки. Хранение данных – очень важная часть программирования и очень полезно осознавать полное устройство операций над числами. Складывая большие и маленькие числа в определённом порядке, можно увеличить точность результата, что зачастую бывает очень важно.

Постановка задачи

Исследовать способы сложения больших дробных чисел с маленькими для достижения оптимального по точности результата.

Руководство пользователя

Так как моя программа является лишь экспериментом, пользовательский интерфейс отсутствует и достаточно лишь запустить программу и она выведет всё необходимое.

На рисунке представлен вывод информации про экспонету. Изначально выводится эталонное значение, вычисленное при помощи математической библиотеки. Далее программа вычисляет значение, складывая результат в прямом, обратном направлении, алгоритмом Кэхэна, в случайном порядке, а также строит графики зависимости ошибки от номера члена. График нормируется в зависимости от максимального значения ошибки, а поэтому отображает только относительную точность в зависимости от шага.

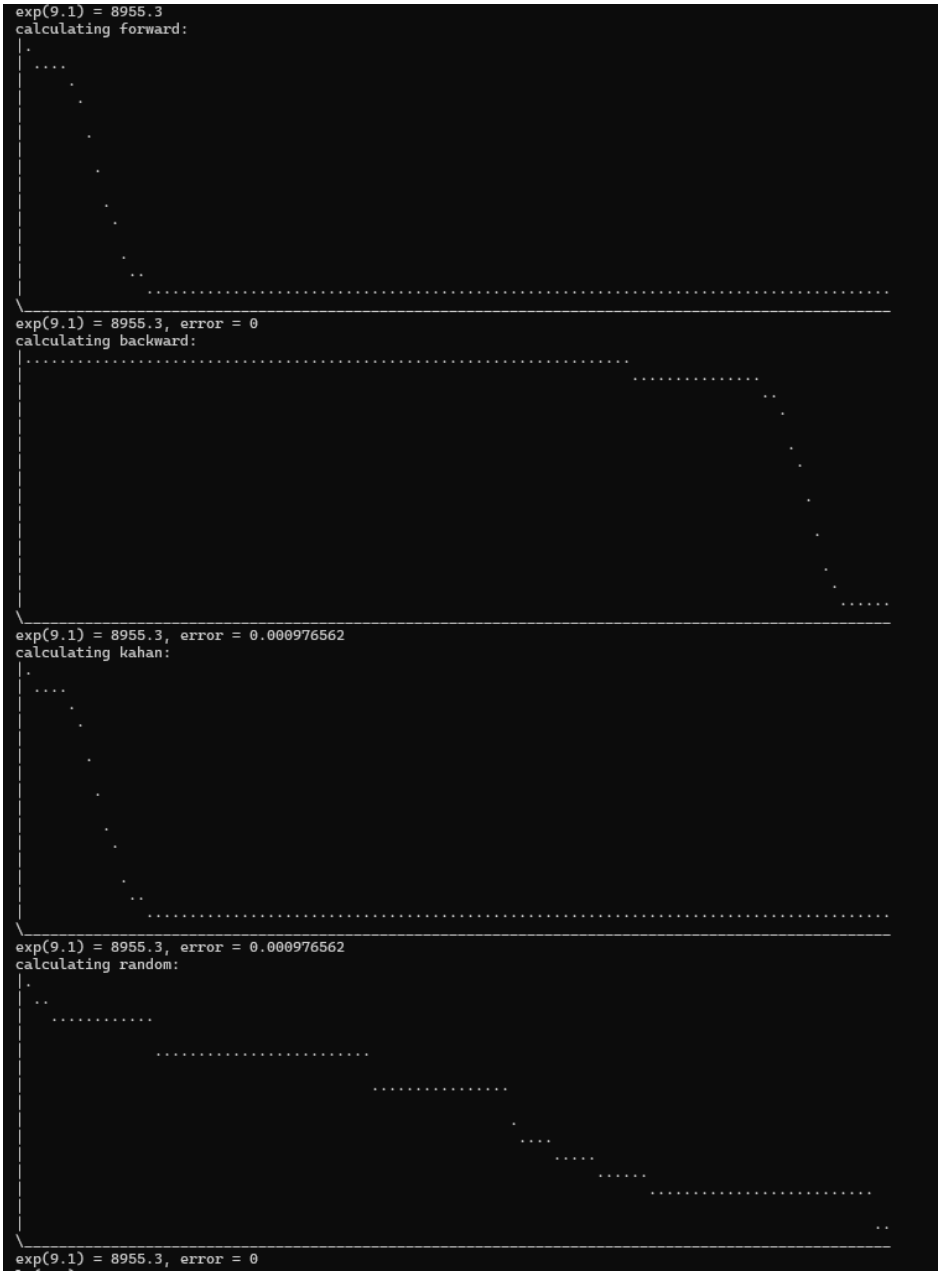


Рисунок 1 Вывод программы для показательной функции

Описание программной реализации

Проект состоит из основного файла и вынесенных отдельно рекуррентных формул вычисления функций синуса, косинуса, экспоненты и логарифма. В основной программе созданы указатели на функции, енумераторы, позволяющие выбрать направление сложения и начальный член, а также структура ряда. В моей реализации ряд полностью определяется своими полями.

```
typedef float (*math_func)(float x);
typedef float (*rec_func)(float xn, int n, float x);
using namespace std;
enum class direction
{
    forward, backward, kahan
};
enum class first_member {
    x, one, x1
};
struct Series
{
    const char* name;
    direction dir;//направление сложения
    math_func orig;//оригинальная функция для сравнения ошибки
    float true_value;//правильное значение
    first_member x0;

    rec_func next;//рекурентная функция ряда
    int n;//размер
    float x;//для какого икса считаем
    float* member;//члены ряда
    float* prev_sum;//префиксная или постфиксная сумма ряда
    float* err;//ошибка на этом шаге
    float res;//результат вычислений
    float r_err;//итоговая ошибка
    Series(const char* name, math_func orig, rec_func next, first_member x0) {
        this->name = name;
        this->orig = orig;
        this->next = next;
        this->x0 = x0;
    }
};
```

Рисунок 2. объявление структуры ряда

Для наполовину заполненного ряда есть функция fill. Она заполняет массив членов и ошибки. (представлен только фрагмент)

```
void fill(Series& s) {//должно быть заполнено всё кроме массивов и ...
    s.true_value = s.orig(s.x);
    //free(s.member);
    //free(s.prev_sum);
    //free(s.err);
    s.member = (float*)malloc(sizeof(float) * s.n);
    s.prev_sum = (float*)malloc(sizeof(float) * s.n);
    s.err = (float*)malloc(sizeof(float) * s.n);
    s.member[0] = (s.x0 == first_member::x) ? s.x : 1;
    if (s.x0 == first_member::x1) {//особенности логарифма
        s.member[0] = s.x - 1;
    }
    for (int i = 1; i < s.n; i++) {
        s.member[i] = s.next(s.member[i - 1], i-1, s.x);
        //cout << s.member[i-1] << " ";
    }
    if (s.dir == direction::forward) {
        s.prev_sum[0] = s.member[0];
        s.err[0] = abs(s.prev_sum[0] - s.true_value);
        for (int i = 1; i < s.n; i++) {
            s.prev_sum[i] = s.prev_sum[i - 1] + s.member[i];
            s.err[i] = abs(s.prev_sum[i] - s.true_value);
            //cout << s.prev_sum[i] << " ";
        }
    }
```

Рисунок 3. Фрагмент функции fill

Также в программе есть возможность отрисовки графиков функции, который рисует ошибку. Для работы ему нужно передать ряд и высоту. Сначала он определит максимальное значение ошибки и сохранит его. Потом создаст двумерный массив размерами n на height – таблицу, на которой будут отмечаться точки. Далее каждый элемент массива ошибки он переносит из промежутка [max_err, 0] на промежуток [height, 0] по формуле из приложения и в таблице ставит точку на соответствующей высоте, а затем выводит таблицу в консоль.

Функция print_table просто заменяет в структуре направление, заполняет её , выводит в консоль ошибку и строит график зависимости ошибки.

В main инициализируется массив структур ряда и вызывается функция вывода.

```
int main()
{
    const int n_of_series = 4;
    Series a[n_of_series] = {
        Series("sin", sin, r_sin, first_member::x),
        Series("cos", cos, r_cos, first_member::one),
        Series("exp", exp, r_exp, first_member::one),
        Series("ln", log, r_ln, first_member::x1)
    };
    const int n_of_x = 5;
    float x[5] = {
        9.1, 1.2, 1.3, 1.4, 0.5
    };
    int n_of_members = 100 ;
    int height = 15;
    print_table(a, n_of_series, x, n_of_x, n_of_members, height);
}
```

Рисунок 4. Функция main

Ряд задаётся названием, функцией из math, реккурентной функцией и начальным членом. Снизу задаётся количество членов и высота графика.

Результаты экспериментов

Я провёл множество экспериментов. При использовании типа double ошибка равна 0, так что исследования проводились на float. При x > 10 члены ряда становятся невероятно большими и из-за этого функции считаются неправильно, ряд начинает расходиться, при этом ошибка начинает выглядеть очень странно.

Рисунок 5. необычное поведение синуса

Для решения этой ошибки можно использовать периодичность синуса и косинуса. Также нельзя точно определить, какой порядок сложения оптимален, в одном из тестов оказалось, что для синуса оптимален прямой, для косинуса алгоритм Кэхэна, а для показательной обратный. А в некоторых случаях самым оптимальным оказывается случайное сложение. В одном эксперименте для экспоненты алгоритм Кэхэна вычислил абсолютно точно, когда все другие способы ошиблись на большое число. Однако из этого всё равно можно сделать вывод, что результат зависит от порядка сложения и на это нужно обращать внимание.

При больших значениях график ошибки экспоненты выглядит очень интересно

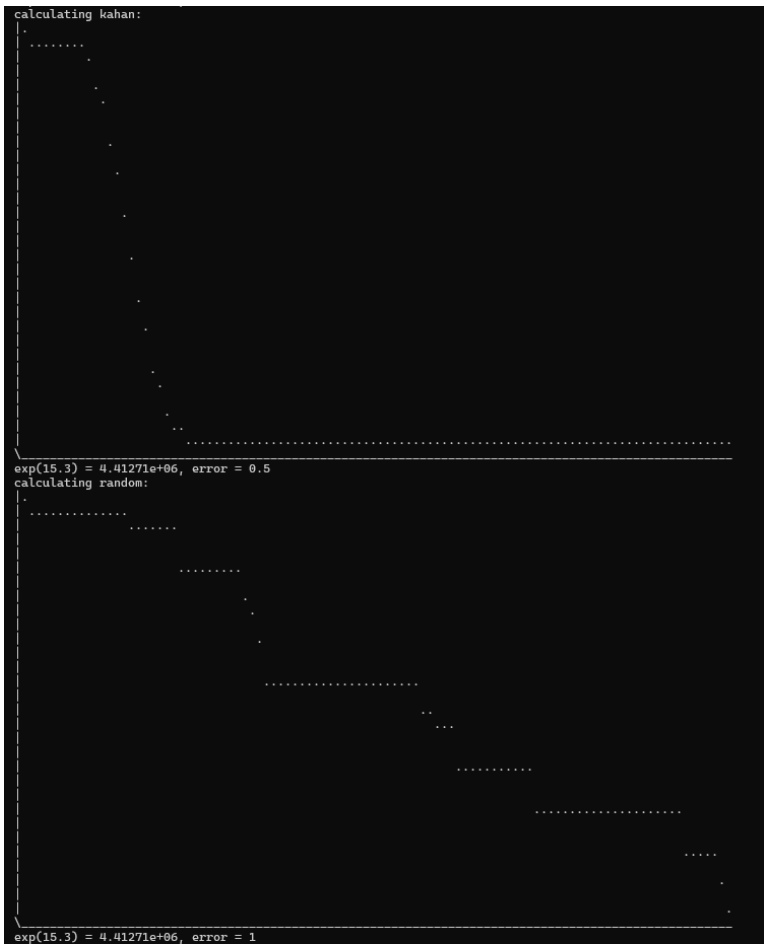


Рисунок 6. Уменьшение ошибки в показательной функции при большом x

Заключение

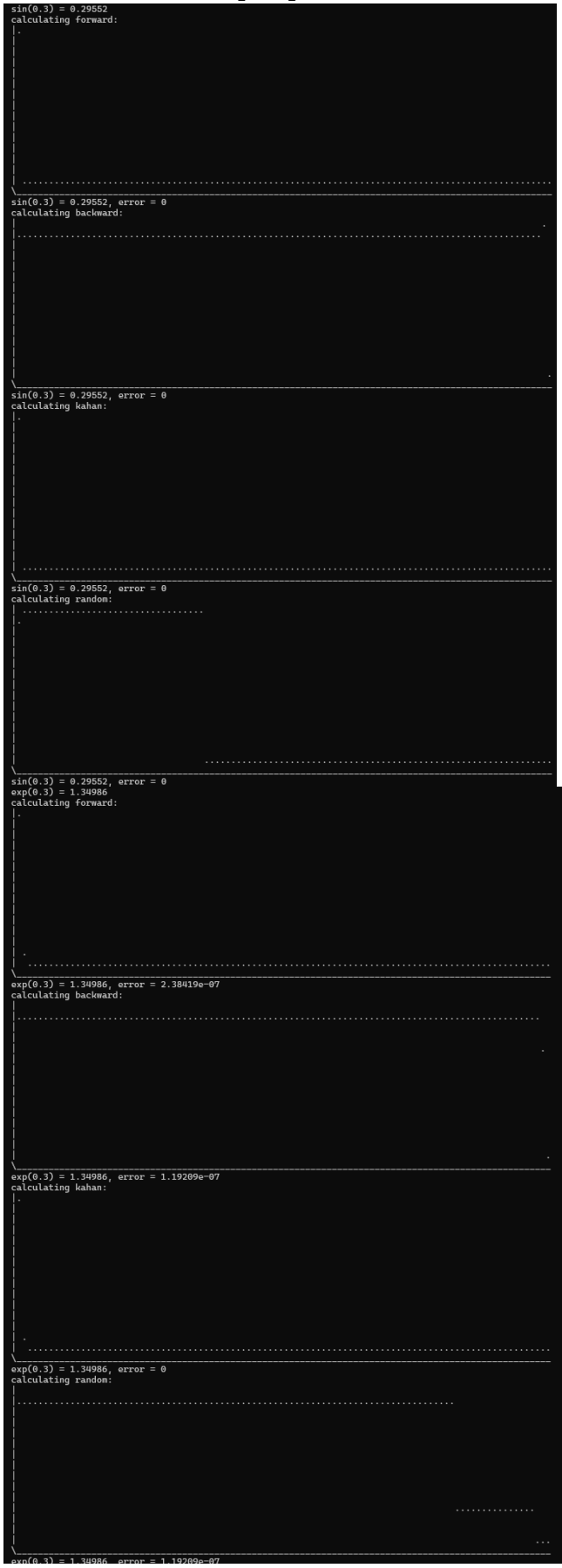
В результате я написал программу для вычисления значений четырёх элементарных функций, используя ряд Тейлора и 4 способа сложения членов ряда. Результаты при вычислении разными способами были почти всегда различными, из чего можно сделать вывод, что порядок сложения влияет на итоговый ответ в c++. Кроме этого, я узнал, что у синуса и косинуса точность повышается нелинейно и в некоторых случаях вычисление меньшего количества членов может дать более точный результат.

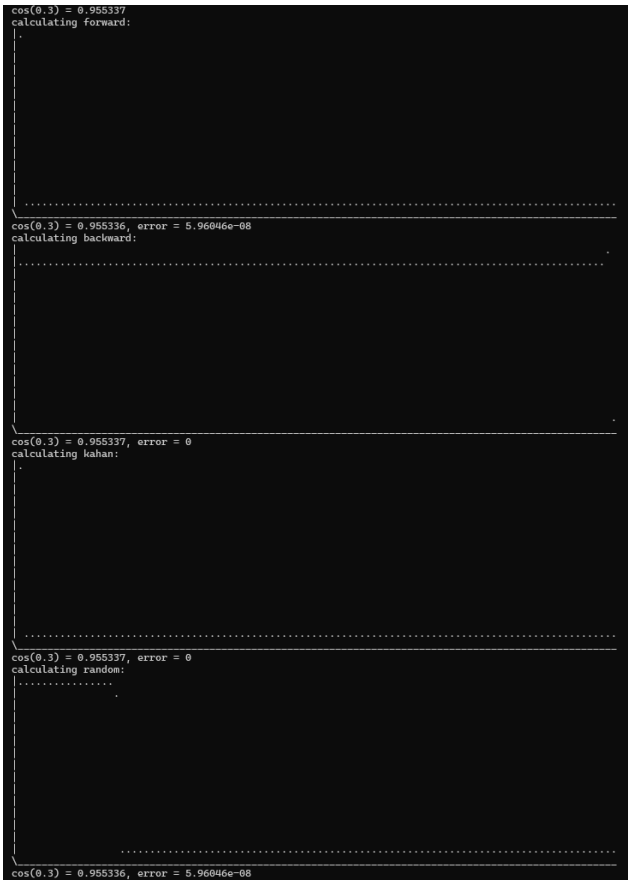
Литература

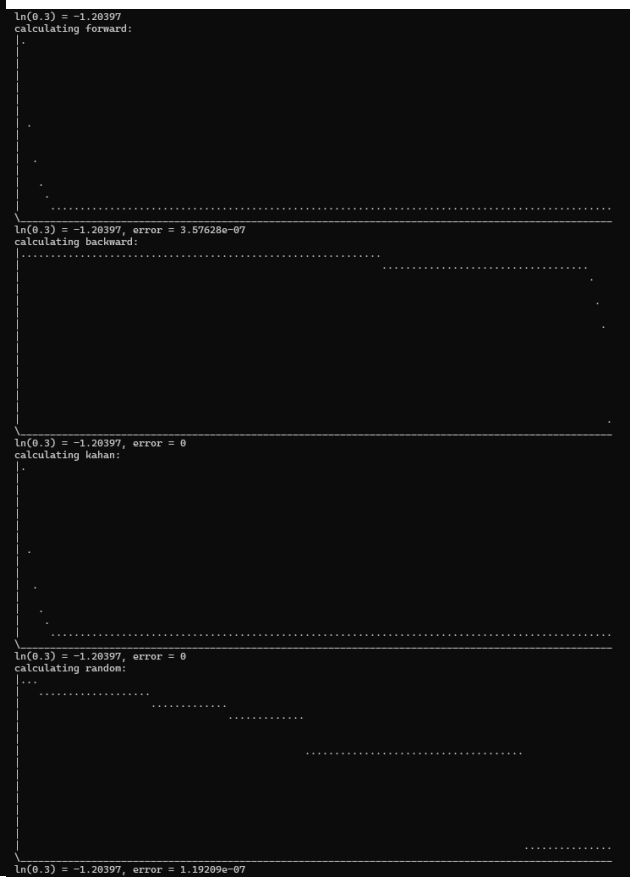
1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.
3. Грэхель, Кнут. Конкретная математика

Приложение

Полный вывод программы для x = 0.3







```
int map(float a, float b, int na, int nb, float x) {
    return (x - a) * (nb - na) / (b - a) + na;
}
```

Рисунок 7. Функция map, которая сжимает массив ошибки